Appearance Manifold Contributor Guide

Runtime interpolation architecture, code ownership, limitations, and improvement roadmap

1. Module Role

AppearanceManifold converts weathered states generated by GammaTon into 7D tensors and runtime trajectory binaries, then interpolates between saved keyframes and applies the resulting textures to a Dynamic Material Instance.

Layer	Responsibility
GammaTon	Editor-focused weathering simulation and PNG generation.
AppearanceManifold	Keyframe tensorization, trajectory generation, runtime interpolation, and material application.
WeatheringManagerComponent	Blueprint orchestration layer for file paths, keyframe management, timer-driven playback, and material updates.

2. Key Source Files

File	Role
AppearanceManifoldBlueprintFunctionLibrary.h/.cpp	Most Blueprint-callable C++ APIs.
BuildNearestTrajectoryAsync.h/.cpp	Async Blueprint node that builds nearest trajectory caches on ThreadPool.
AppearanceManifold.cpp	Module startup, Python script path registration, and Python wheel dependency checks.
Scripts/MainManager.py	Appearance Manifold trajectory generation entry point.
Scripts/Modules/Utility.py	Runtime binary export utilities.

3. 7D Tensor Data Model

Channel	Meaning
0, 1, 2	BaseColor R, G, B
3, 4, 5	Specular R, G, B
6	Roughness
Offset	Pixel i begins at i * 7.

4. Binary Format and Paths

> Current implementation: LoadWeatheringBinaryData reads [T:int32][float * T * 7]. Older notes describing a [T][N] header refer to a broader export format and should not be used for the current runtime loader.

Data	Path / Format
TensorCache	Content/TensorCache/[ActorName]_[Index].bin stores per-keyframe 7D tensors.
Trajectory	Content/WeatheringResults stores runtime trajectory samples.
GammaTon source	Saved/GammaTon/Manifold stores Base/Spec/Rough PNG sources.

5. Blueprint Orchestration

Node / Event	Implementation Role
Set Key Frame Textures	Combines Base/Spec/Rough Texture2D inputs into a 7D tensor and saves it at the current count index.

Node / Event	Implementation Role
Set Key Frame Count With Files	Counts contiguous TensorCache files starting from index 0.
Get New Key Frame	Loads GammaTon PNG results and saves the next keyframe tensor.
Clear Key Frame Texture	Deletes old cache files and re-registers sample textures as keyframe 0.
Generate Weathering Trajectory	Reconstructs the middle keyframe and passes it to the Python pipeline.
Apply Weathered Texture	Runs nearest cache generation, interpolation, texture reconstruction, and MID parameter updates.

6. Runtime Call Chain

1. BeginPlay calls Set Key Frame Count With Files.
2. If Allow Weathering && KeyWeatheredTexturesCount > 1, the component waits 2 seconds and loads the trajectory binary.
3. Trajectory Point Count and Trajectory Samples are stored as component state.
4. Initial tensors for keyframes 0 and 1 are loaded and applied.
5. A looping Weathering Step Up timer is started using Weathering Speed as the interval.
6. Weathering Step Up calls Get Key Index to map the global step to segment index and Alpha, then applies the current pair.

7. Apply Weathered Texture Details

1. BuildNearestTrajectoryCacheAsync(Tex A 7D, Trajectory Samples, T).
2. BuildNearestTrajectoryCacheAsync(Tex B 7D, Trajectory Samples, T).
3. InterpolateWeatheringCached(Tex A, Tex B, Cached A, Cached B, Trajectory, T, Alpha).
4. ReconstructTexturesFromTensor(Result, Sample Size, Existing Base/Spec/Rough).
5. Get Owner -> Get Component By Class(MeshComponent) -> Is Valid.
6. ApplyWeatheringTexturesToMesh(MaterialIndex=0, ParamNames=Weathering_Base, Weathering_Spec, Weathering_Rough, ExistingMID=OutMID).

> Performance note: The current Blueprint rebuilds nearest caches for A and B on each Apply call. Reusing caches while the keyframe pair is unchanged is the highest-value optimization.

8. C++ API Behavior

Function	Behavior
CombineTextureSources	Reads Base/Spec/Rough UTexture2D inputs and combines them into one 7D float array.
BuildNearestTrajectoryCache	Brute-force nearest sample search per pixel. O(PixelCount * T * 7).
InterpolateWeatheringCached	Computes an intermediate 7D tensor from nearest index caches and Alpha.
ReconstructTexturesFromTensor	Creates or reuses three transient UTexture2D objects and updates them through UpdateTextureRegions.
ApplyWeatheringTexturesToMesh	Creates or reuses a MID and assigns texture parameters.

9. Current Limitations

- Nearest search is brute-force, so cost grows quickly with trajectory length and texture resolution.
- Interpolation uses rounded trajectory indices and is not fully continuous between adjacent samples.
- Material Index 0 and parameter names are currently hardcoded in the Blueprint.
- The first MeshComponent found on the owner is used, which is limiting for actors with multiple mesh components.
- Frequent transient texture uploads can cause frame hitches at high resolution.
- The editor Python pipeline and runtime loading path should be separated more explicitly for packaging.

10. Improvement Roadmap

Area	Recommendation
Cache reuse	Store nearest index caches per keyframe pair and reuse them while only Alpha changes.

Area	Recommendation
Search acceleration	Evaluate KD-tree, quantized LUT, or trajectory segment acceleration structures.
Material flexibility	Expose Material Index and parameter names as component variables.
GPU path	Evaluate Compute Shader or Render Target based reconstruction paths.
Binary versioning	Add version, resolution, channel count, and format metadata.
Validation	Return clearer errors for missing files, tensor length mismatch, and texture size mismatch.